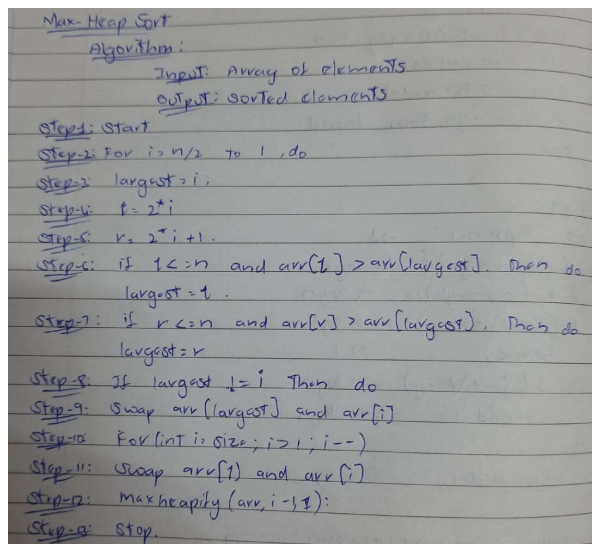


PRACTICAL-03

AIM: Implementation of max-heap sort algorithm.

1.) MAX HEAP

ALGORITHM:



Max-Heap Sort
Algorithm:
Input: Array of elements
Output: Sorted elements

Step-1: Start
Step-2: For $i = n/2$ to 1, do
Step-3: largest = i ;
Step-4: $l = 2 * i$
Step-5: $r = 2 * i + 1$;
Step-6: if $l \leq n$ and $arr[l] > arr[largest]$, then do
largest = l ;
Step-7: if $r \leq n$ and $arr[r] > arr[largest]$, then do
largest = r ;
Step-8: If largest $\neq i$ Then do
Step-9: Swap $arr[largest]$ and $arr[i]$
Step-10: For (int $j = size$; $j > 1$; $j--$)
Step-11: Swap $arr[j]$ and $arr[i]$
Step-12: maxheapify($arr, i-1$);
Step-13: Stop.

PROGRAM:

```
#include <iostream>
```

```
using namespace std;
```

```
void maxheapify(int arr[], int size, int i)
```

```
{  
    int largest = i;  
    int l = 2 * i;  
    int r = 2 * i + 1;  
  
    if (l <= size && arr[l] > arr[largest])  
    {  
        largest = l;  
    }  
}
```

```

    if (r <= size && arr[r] > arr[largest])
    {
        largest = r;
    }

    if (largest != i)
    {
        swap(arr[largest], arr[i]);
        maxheapify(arr, size, largest);
    }
}

void heapsort(int arr[], int size)
{
    for (int i = size / 2; i >= 1; i--)
    {
        maxheapify(arr, size, i);
    }
    for (int i = size; i > 1; i--)
    {
        swap(arr[1], arr[i]);
        maxheapify(arr, i - 1, 1);
    }
}

int main()
{
    int n;
    cout << "Enter number of elements: ";
    cin >> n;
    int arr[n + 1];
    cout << "Enter elements: ";
    for (int i = 1; i <= n; i++)
    {
        cin >> arr[i];
    }
    heapsort(arr, n);
}

```

```

cout << "Sorted array: ";

for (int i = 1; i <= n; i++)
{
    cout << arr[i] << " ";
}

return 0;
}

```

OUTPUT:

```

Output
Enter number of elements: 5
Enter elements: 5
9
7
6
1
Sorted array: 1 5 6 7 9
=== Code Execution Successful ===

```

TIME COMPLEXITY:

Time Complexity:

for ($i = n/2$; $i >= 1$; $i--$) $\rightarrow n$
 maxHeapify(arr, n, i) $\rightarrow \log n$
 for ($i = n$; $i > 1$; $i--$) $\rightarrow n$
 swap(arr[i], arr[1]) $\rightarrow 1$
 MaxHeapify(arr, i-1, 1) $\rightarrow \log n$

$T(n) = n \times \log n + n + 1 \times \log n$
 $= n \log n + n \log n$
 $T(n) = \Theta(n \log n)$

Best case: $T(n) = O(n \log n)$
Average case: $T(n) = O(n \log n)$
Worst case: $T(n) = \Theta(n \log n)$

1) Best case:
 for $i = n/2$ to 1
 maxHeapify(arr, n, i) $\rightarrow n$
 for $i = n$ to 2
 swap(arr[i], arr[1])
 maxHeapify(arr, i-1, 1) $\rightarrow \log n$
 $T(n) = O(n) + n \times O(\log n)$
 $T(n) = O(n) + O(n \log n)$
 $T(n) = O(n \log n)$

2) Average case $\rightarrow n$
 for $i = n$ to 2
 swap(1, i)
 maxHeapify(1) $\rightarrow \log n$
 $T(n) = \Theta(n) + 1 \times \log n$
 $T(n) = O(n) + O(n \log n)$
 $T(n) = O(n \log n)$

if Worst case
 maxHeapify from root to lowest level of the heap
 $\rightarrow \log_2 n$
 maxHeapify(1) $\rightarrow \log n$
 $T(n) = n + n \times \log n$
 $T(n) = O(n \log n)$